Ömer Faruk Akbay

22102584 HW2

1)

A)

Parameters for first trial:

N_hidden = 64

batch_size = 32

learning_rate = 0.1

epochs = 100

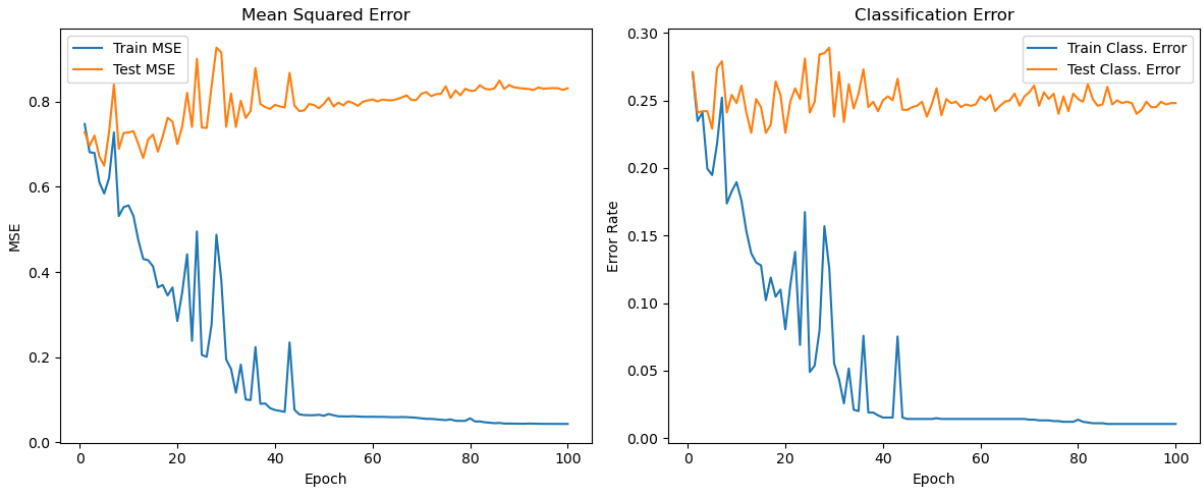


Figure 1: Trial 1

The model is highly overfitting, so I reduced the number of epochs and increased the batch size.

Parameters for the second trial:

N_hidden = 64

batch_size = 64

learning_rate = 0.1

epochs = 50

Train MSE: 0.1253 - Test MSE: 0.7504

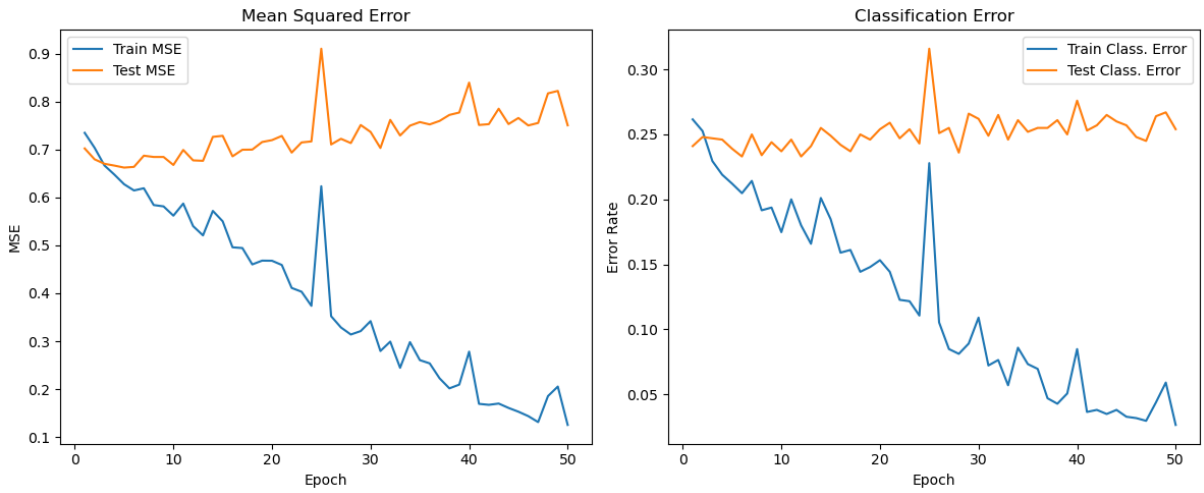


Figure 2: Trail 2

Overfitting decreases, but we still see it. I want to see how the learning rate will affect the MSE and classification error.

Parameters for the third trial:

N_hidden = 64

batch_size = 64

learning_rate = 0.3

epochs = 50

Epoch 50/50 - Train MSE: 0.1019 - Test MSE: 0.8304

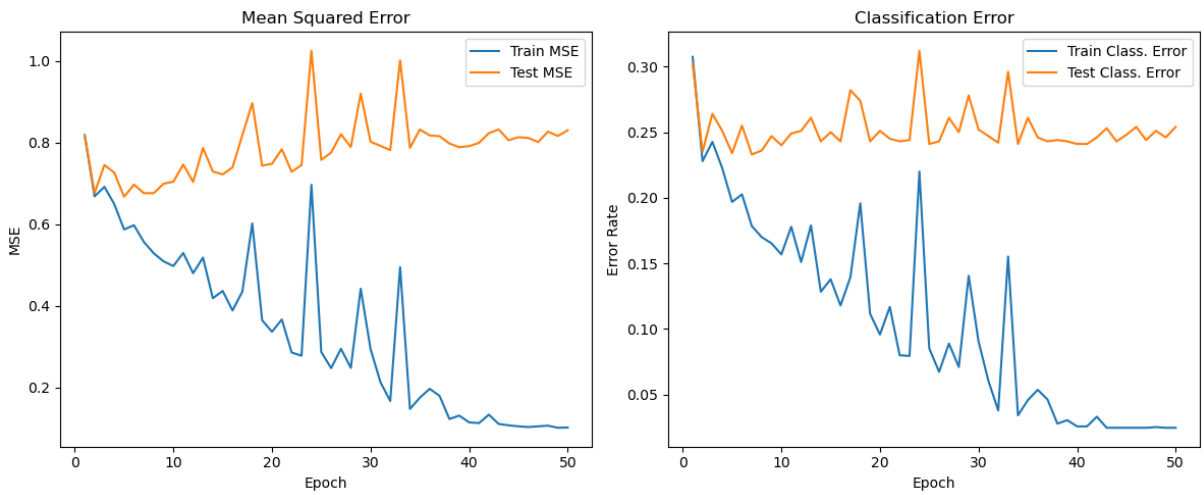


Figure 3: Trial 3

Still, we have overfitting. Let's reduce complexity by reducing the number of hidden layers and the epoch size.

Parameters for the fourth trial:

N_hidden = 16

batch_size = 64

learning_rate = 0.3

epochs = 30

Epoch 30/30 - Train MSE: 0.1590 - Test MSE: 0.7764

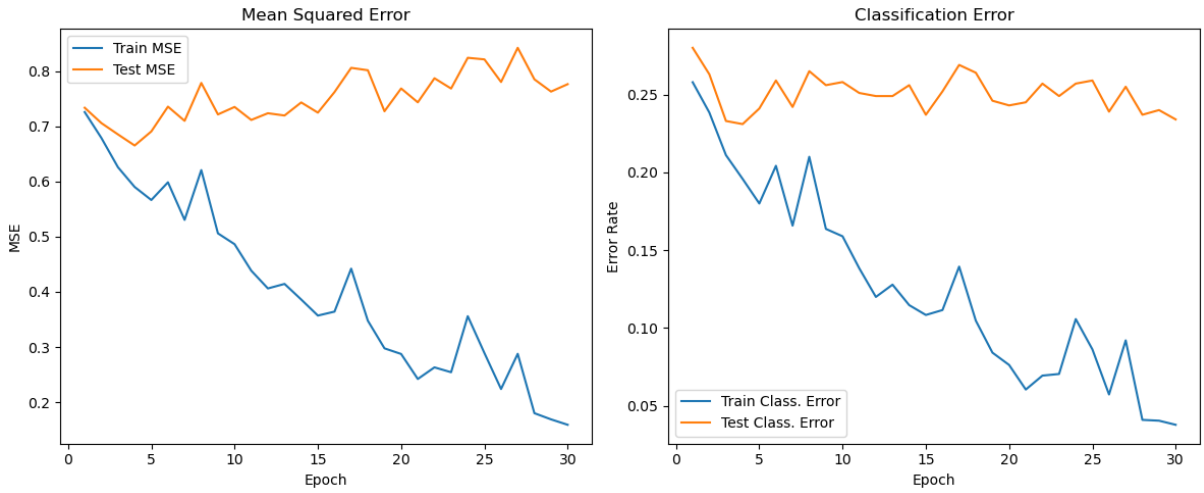


Figure 4: Trial 4

There is still a gap between test and train error; let's even further reduce the epoch size and learning rate.

Parameters for the fifth trial:

N_hidden = 16

batch_size = 64

learning_rate = 0.1

epochs = 8

Epoch 8/8 - Train MSE: 0.5678 - Test MSE: 0.6606

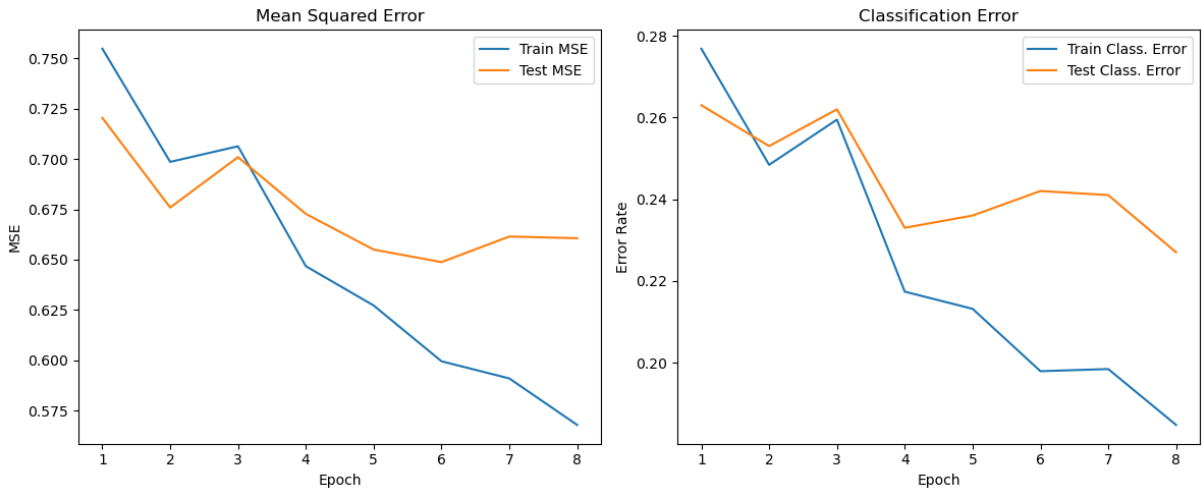


Figure 5: Trail 5 (best)

In the end, we eliminated overfitting by reducing the number of hidden layers and the epoch size, and increasing the batch size.

B)

Training Set: Both the Mean Squared Error (MSE) and the Classification Error consistently decrease as the number of epochs increases. This shows the network is successfully learning and minimizing the loss function on the data it sees.

Testing Set: The testing metrics initially follow the downward trend of the training metrics, indicating that the model is learning generalizable features. However, around epoch 6, the Test MSE begins to slightly increase (from 0.6488 to 0.6606), while the Test Classification Error fluctuates. This divergence is the classic signature of early overfitting, where the network begins to fit the noise in the training data rather than the underlying pattern. Stopping at epoch 8 was a good call to prevent this from worsening.

The MSE is not a perfect predictor of classification error since, while the MSE is continuous, the classification error is discrete. A model can reduce MSE by being more confident in already correct classifications, but classification error will remain the same. On the other hand, a few vastly incorrect outlier predictions will skew the MSE much higher, even if the overall percentage of correct classifications remains relatively stable.

C)

Training N_low = 4...

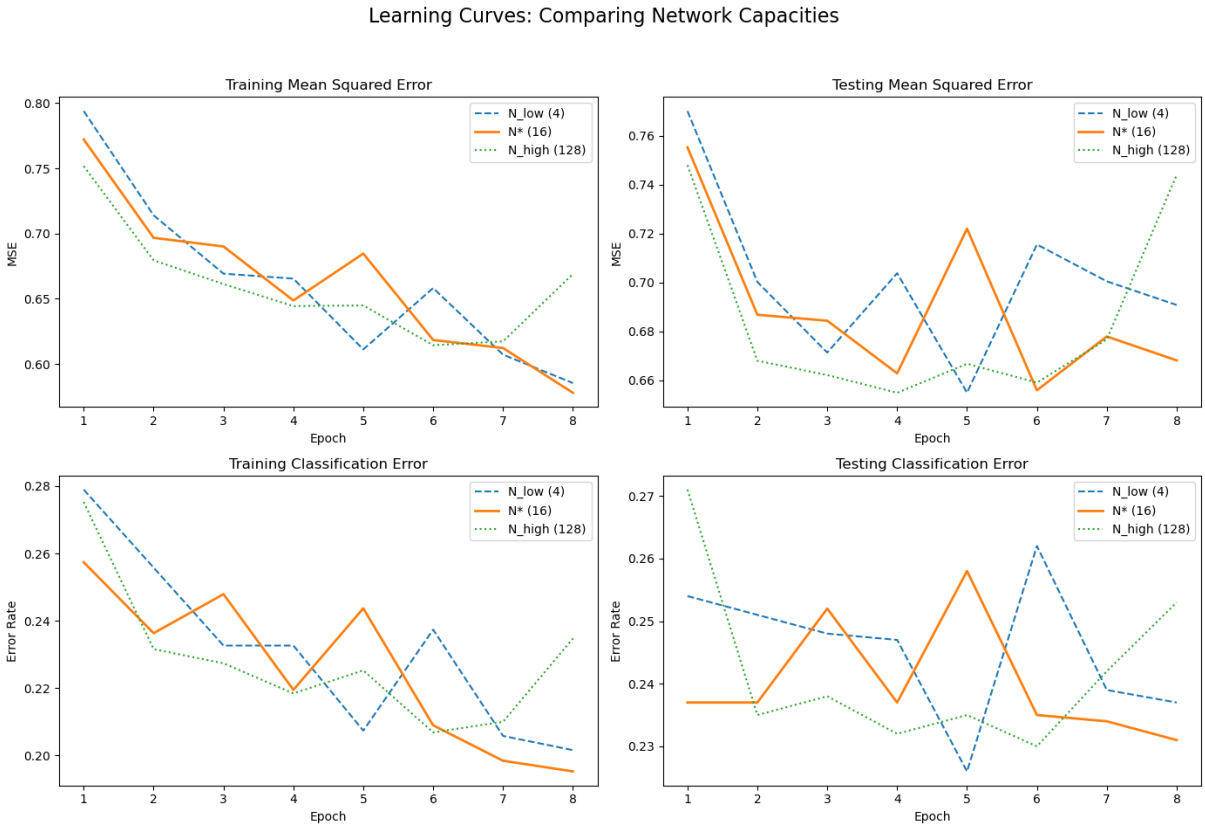
Train MSE: 0.5853 - Test MSE: 0.6908

Training N_star = 16...

Epoch 8/8 - Train MSE: 0.5778 - Test MSE: 0.6681

Training N_high = 128...

Epoch 8/8 - Train MSE: 0.6690 - Test MSE: 0.7440



N_low: Struggles to minimize the testing error as effectively as the others. This is underfitting. The network doesn't have enough parameters to learn the complex features required to distinguish cats from cars.

N_star: Offers the best balance. It minimizes the training error reasonably well while keeping the test error tightly correlated.

N_high: Will likely show the lowest Training MSE, but its Testing MSE will diverge and spike higher than N_star. This clearly illustrates overfitting, as the excessive number of parameters memorizes the training data noise.

D)

Parameters:

H1_size = 16

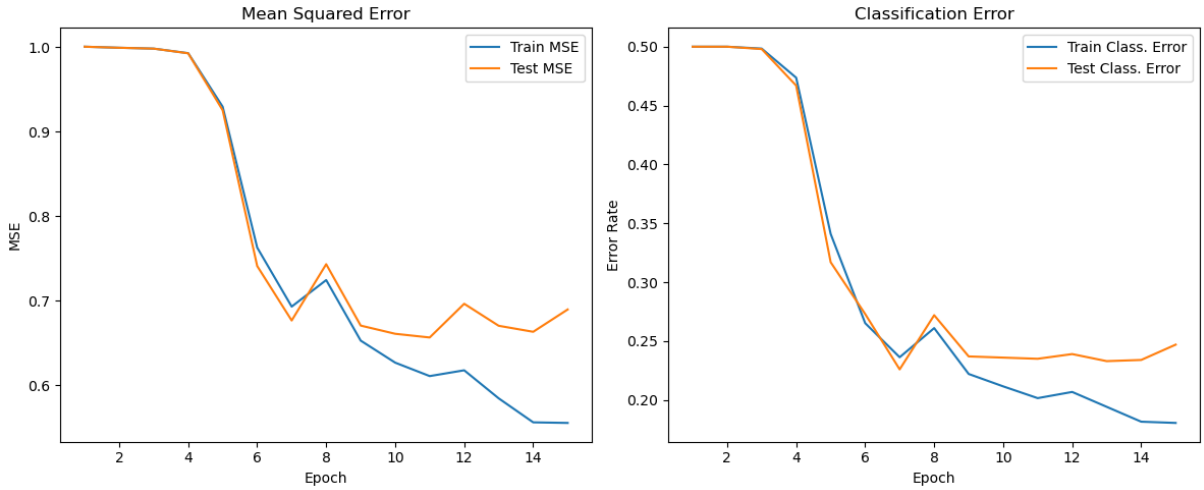
H2_size = 8

batch_size = 64

learning_rate = 0.1

epochs = 16

Train MSE: 0.5561 - Test MSE: 0.663



Convergence: Deeper networks often take slightly longer to converge than a shallow single-layer network. We notice that the error stays relatively flat for the first few epochs before dropping. This happens because the gradients have to backpropagate through more layers, and the weights need more time to align.

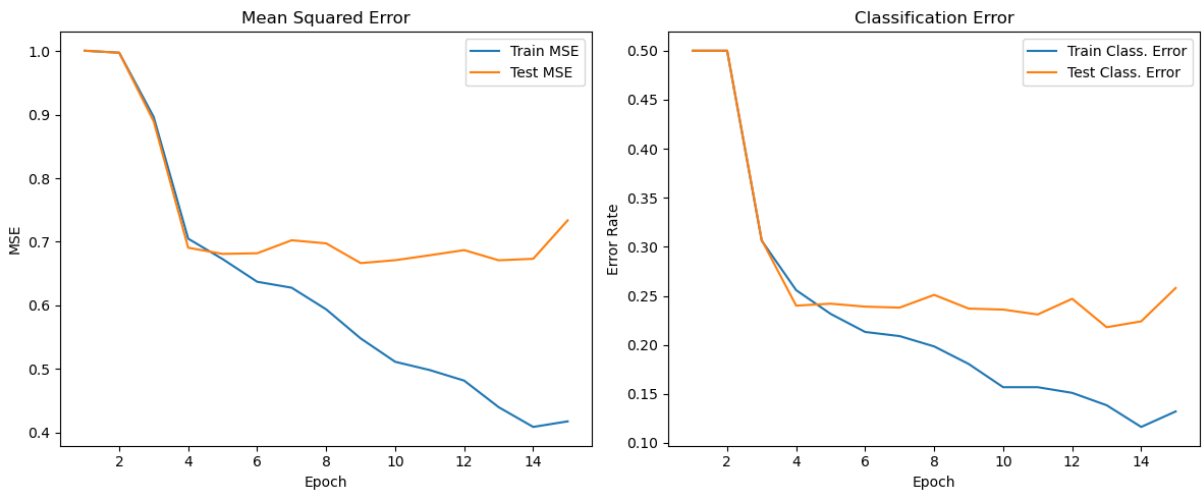
Classification Performance: Since the dataset is relatively simple, there was no huge change in classification performance.

Overfitting Risk: With more layers, there are more parameters. We observe that the test MSE diverges from the train MSE earlier and more aggressively than it did with your single-layer model, so we need to rearrange the layer sizes accordingly.

E)

Momentum; alpha=0.5

Train MSE: 0.4087 - Test MSE: 0.6733



Convergence Behavior: The network with momentum should converge noticeably faster in the early epochs. We see steeper initial drops in the MSE and Classification Error curves compared to part (d).

Stability: The learning curves, especially for the test set, might appear smoother. Because the momentum term averages out the mini-batch gradient noise, the path to the minimum is more direct.

Overfitting Acceleration: Because it learns faster, we are on the border of overfitting, but these parameters work well.

2)

A)

I used early stopping to prevent overfitting.

D,P = 32,256

Epoch 28

Train Loss: 2.3701 - Val Loss: 2.5603

D,P = 16,128

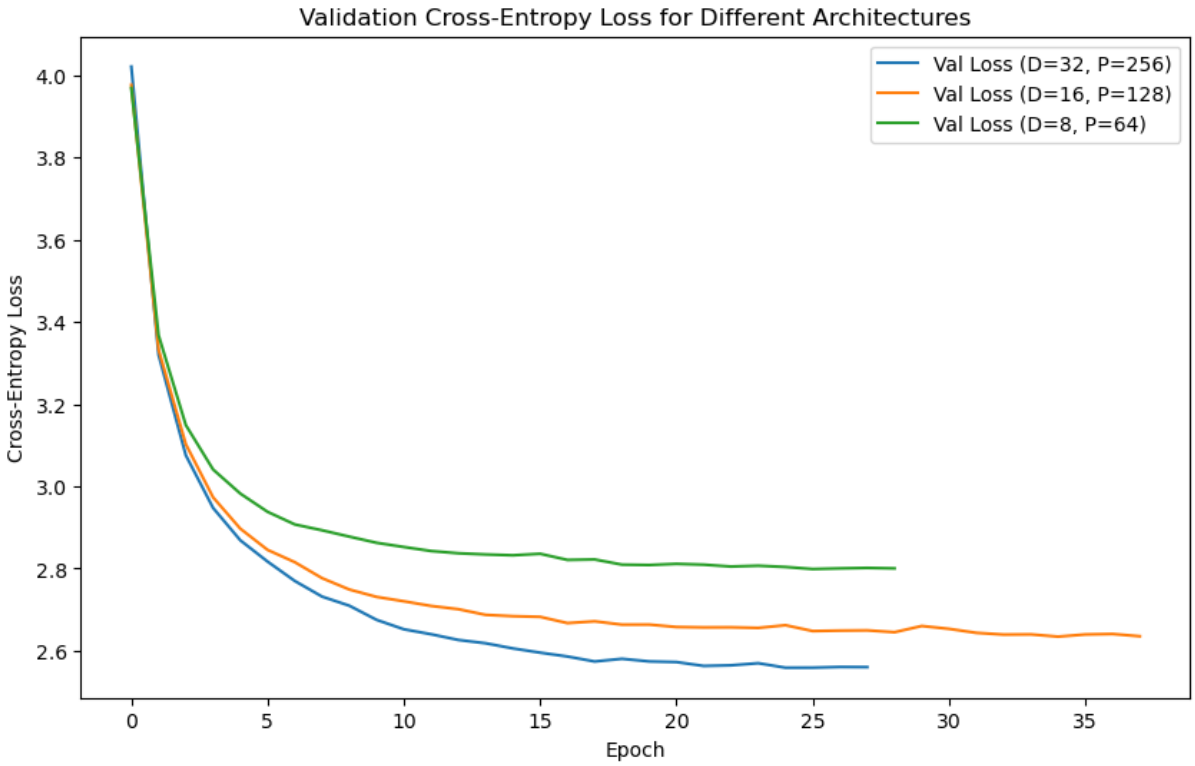
Epoch 38

Train Loss: 2.5292 - Val Loss: 2.6353

D,P = 8,64

Epoch 29

Train Loss: 2.7556 - Val Loss: 2.8005



1. Overall Performance and Learning Capacity

The results clearly indicate that increasing the model's representational capacity both in the embedding space and in the hidden layer leads to better performance.

D=32, P=256 (Highest Capacity): This model achieved the lowest final validation. A larger embedding size allows the network to learn richer, more nuanced semantic relationships between the 250 vocabulary words. The larger hidden layer provides the necessary parameters to model complex sequences of these dense representations.

D=16, P=128 (Medium Capacity): This model achieved a higher validation loss and took longer to trigger early stopping, suggesting it struggled to optimize further.

D=8, P=64 (Lowest Capacity): This model hit a performance wall very early, settling on a high validation loss and stopping at epoch 29. An embedding size of 8 prevents learning the words, and it stays superficial.

2. Convergence Speed

The high-capacity model showed the steepest initial learning curve. In the first 5 epochs, its validation loss dropped from 4.02 to 2.86.

The lowest-capacity model took significantly longer to reach similar loss levels, and ultimately plateaued at a much higher level. This suggests that the constrained parameter space makes it much more difficult to find an optimal path down the gradient.

3. Generalization vs. Overfitting

The early stopping mechanism successfully prevented severe overfitting in all three cases. Instead of the validation loss sharply increasing, the models simply plateaued. This indicates that, for this specific, relatively simple NLP task with a small vocabulary, our main challenge is underfitting rather than overfitting.

B)

Top 10 Predictions for 5 Sample Trigrams:

Sample 1:

Input Trigram: 'not next season'

Actual 4th Word: ','

Top 10 Predicted Candidates:

1. . | Prob: 0.8943
2. , | Prob: 0.0440 <--- ACTUAL
3. and | Prob: 0.0149
4. ? | Prob: 0.0133
5. 's | Prob: 0.0067
6. is | Prob: 0.0033
7. for | Prob: 0.0024
8. to | Prob: 0.0021
9. ; | Prob: 0.0018
10. now | Prob: 0.0017

Sample 2:

Input Trigram: 'do not work'

Actual 4th Word: 'as'

Top 10 Predicted Candidates:

1. . | Prob: 0.3985
2. for | Prob: 0.0980
3. , | Prob: 0.0829

4. out | Prob: 0.0784

5. that | Prob: 0.0459

6. with | Prob: 0.0388

7. at | Prob: 0.0321

8. well | Prob: 0.0242

9. ? | Prob: 0.0241

10. here | Prob: 0.0142

Sample 3:

Input Trigram: 'never use it'

Actual 4th Word: 'in'

Top 10 Predicted Candidates:

1. . | Prob: 0.5350

2. , | Prob: 0.0661

3. now | Prob: 0.0545

4. to | Prob: 0.0474

5. at | Prob: 0.0311

6. any | Prob: 0.0272

7. for | Prob: 0.0221

8. in | Prob: 0.0215 <--- ACTUAL

9. right | Prob: 0.0207

10. that | Prob: 0.0136

Sample 4:

Input Trigram: 'one could do'

Actual 4th Word: 'this'

Top 10 Predicted Candidates:

1. it | Prob: 0.4615

2. that | Prob: 0.2448

3. . | Prob: 0.1347

4. the | Prob: 0.0237

5. this | Prob: 0.0237 <--- ACTUAL

6. so | Prob: 0.0098

7. , | Prob: 0.0095

8. what | Prob: 0.0091

9. more | Prob: 0.0075

10. a | Prob: 0.0064

Sample 5:

Input Trigram: 'going to do'

Actual 4th Word: '?'

Top 10 Predicted Candidates:

1. it | Prob: 0.2116

2. . | Prob: 0.1977

3. that | Prob: 0.1283

4. ? | Prob: 0.0802 <--- ACTUAL

5. with | Prob: 0.0596

6. , | Prob: 0.0466

7. about | Prob: 0.0330

8. what | Prob: 0.0283

9. this | Prob: 0.0195

10. for | Prob: 0.0191

Even with a highly constrained vocabulary of only 250 words and a relatively simple architecture, the network has clearly learned underlying rules of English grammar and syntax rather than just memorizing random word strings.

For example, the network successfully predicts the correct grammatical objects to follow specific verbs. In Sample 4 ('one could do'), the top two predictions are 'it' (46.1%) and 'that' (24.4%), which perfectly act as the direct object of the verb "do". The actual word 'this' is also caught in the top 5. In Sample 5 ('going to do'), it repeats this logical behavior, again predicting 'it' and 'that' as the most likely continuations.

Also, the model correctly identifies when a trigram represents a complete thought, and it should finish the sentence. In Sample 1 ('not next season'), the model assigns a massive 89.4% probability to a period (.). Since "not next season" is often put at the end of a sentence . It also places the actual word (,) as the second-most-likely candidate. Similarly, in Sample 3 ('never use it'), the model correctly assumes the phrase is finished, predicting a period (.) with 53.5% confidence.

The network learned which words naturally pair together in English.

In Sample 2 ('do not work'). Even though the actual word ('as') didn't make the top 10, the network's predictions are arguably better and more common! It predicts 'out' (do not work out), 'for' (do not work for), 'well' (do not work well), and 'here' (do not work here). These are highly accurate, context-aware predictions.

In Sample 5 ('going to do'), it successfully predicts question marks (?), recognizing that this trigram frequently appears at the end of interrogative sentences (e.g., "What are we going to do?").

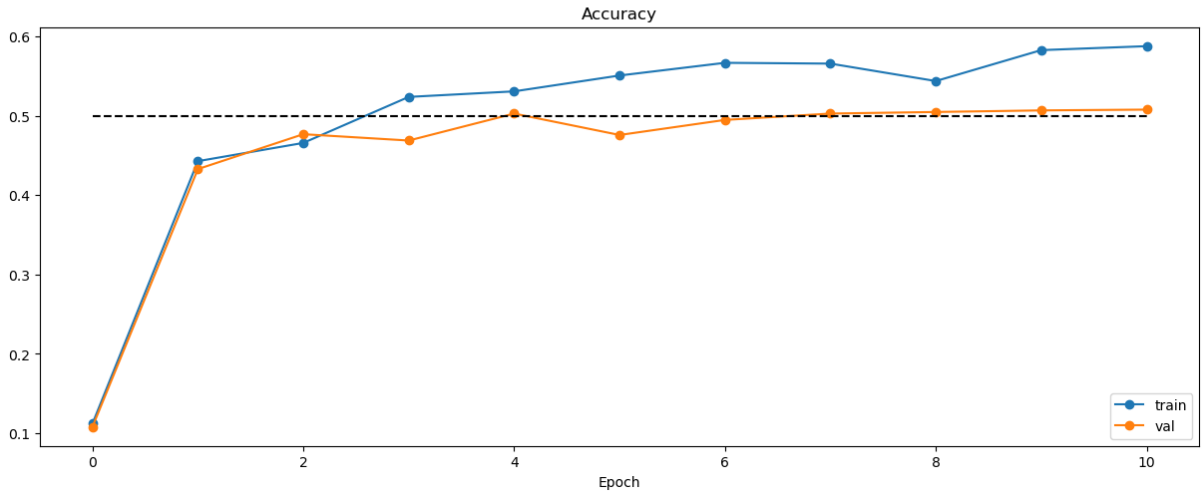
We can conclude that the model's predictions are sensible because the top candidates consistently form grammatically valid, logically sound English sentences. The fact that the network's top predictions are often *more* common English phrases than the "actual" 4th word in the dataset shows that the model effectively generalized the language distribution rather than overfitting to the specific training text.

3-

A)

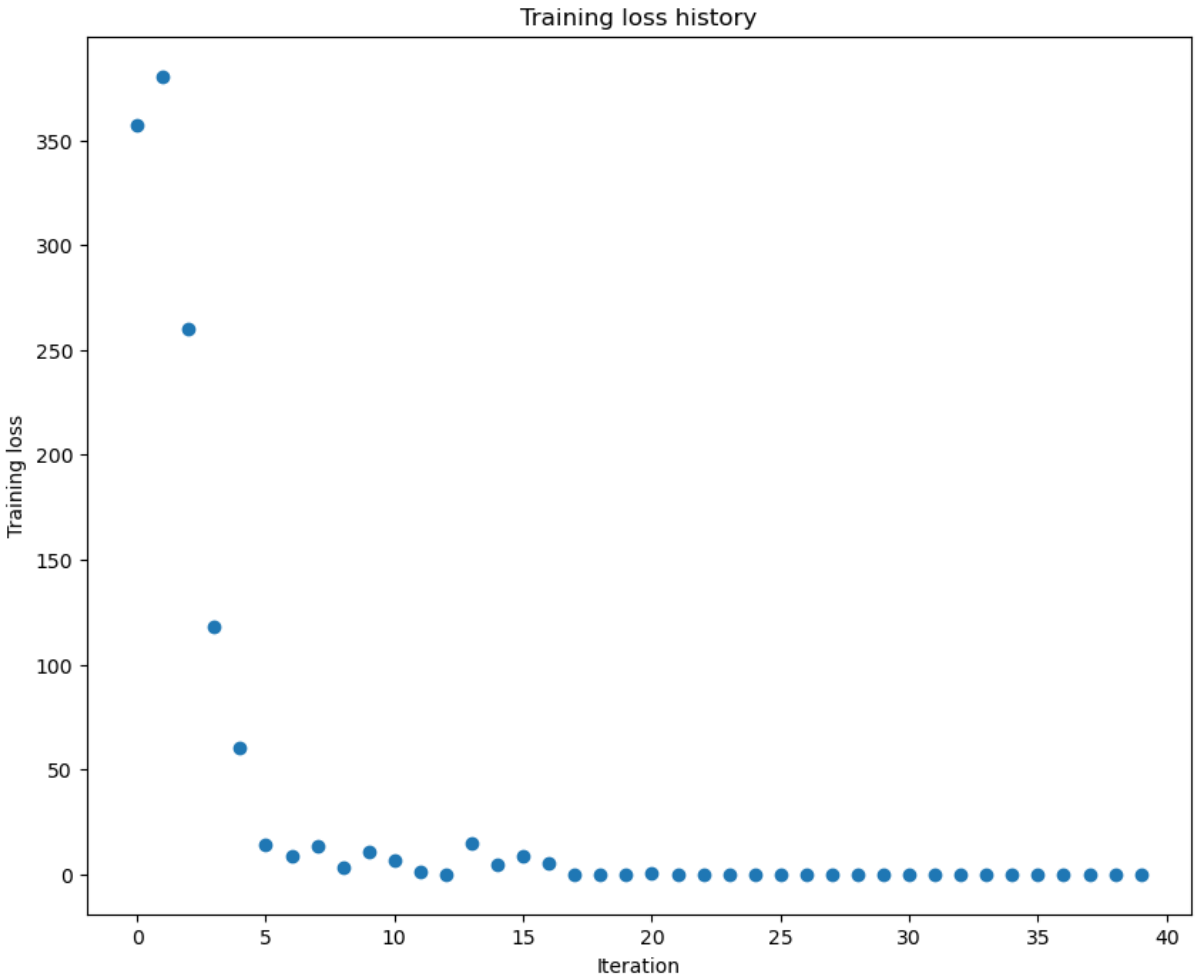
With solver:

(Epoch 10 / 10) train acc: 0.588000; val_acc: 0.508000



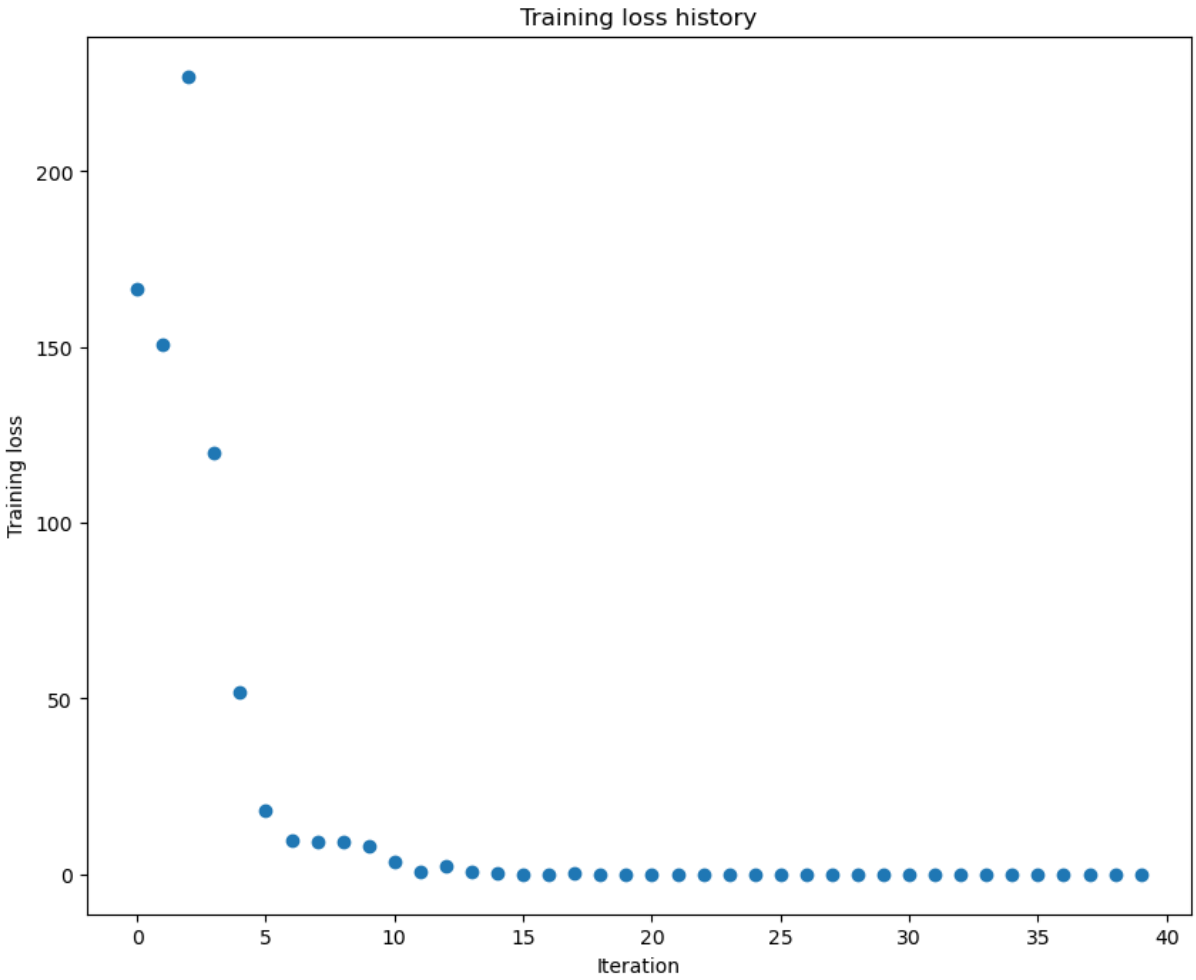
Use a three-layer Net to overfit 50 training examples:

(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.158000



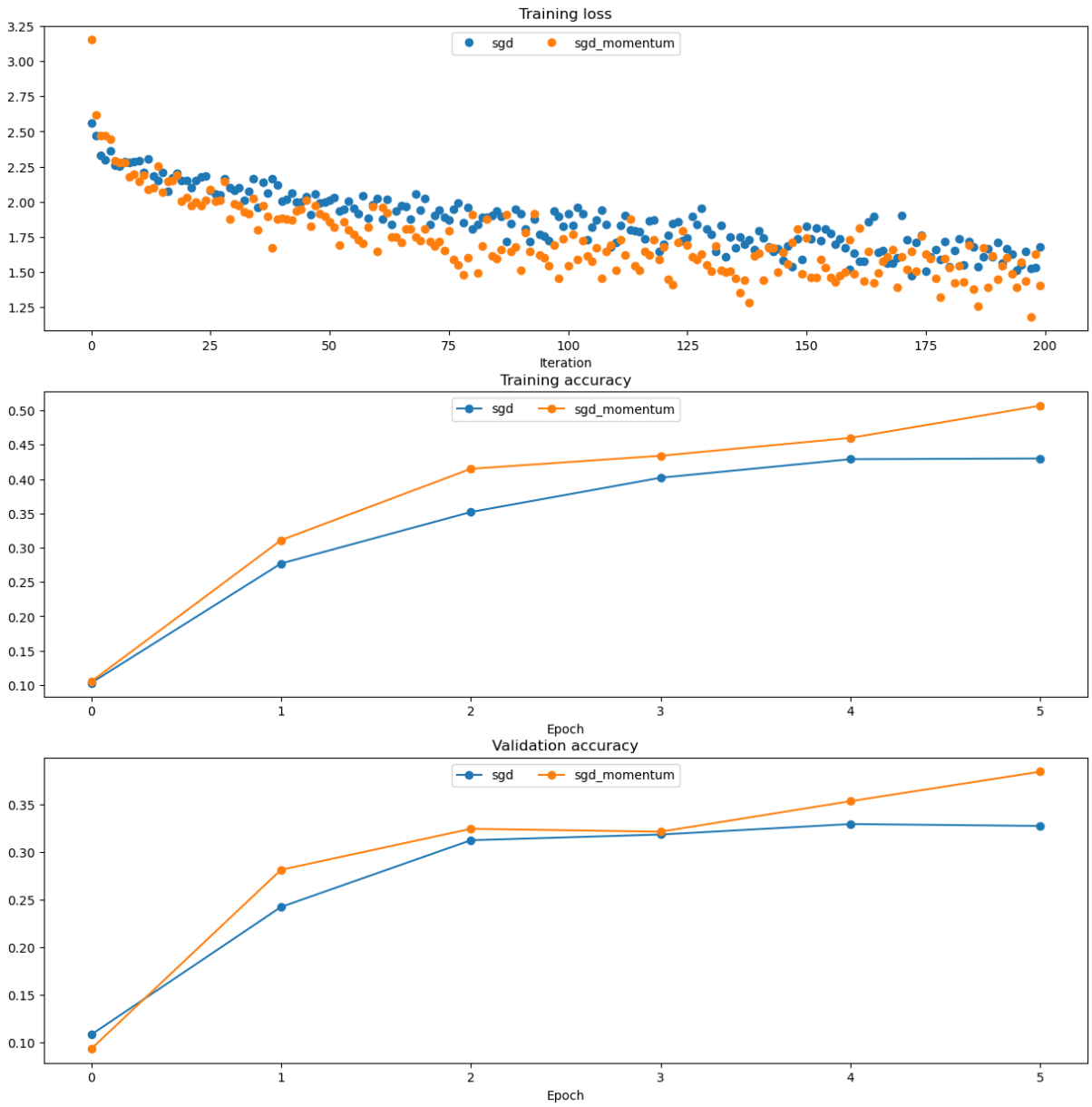
Use a five-layer Net to overfit 50 training examples:

(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.121000



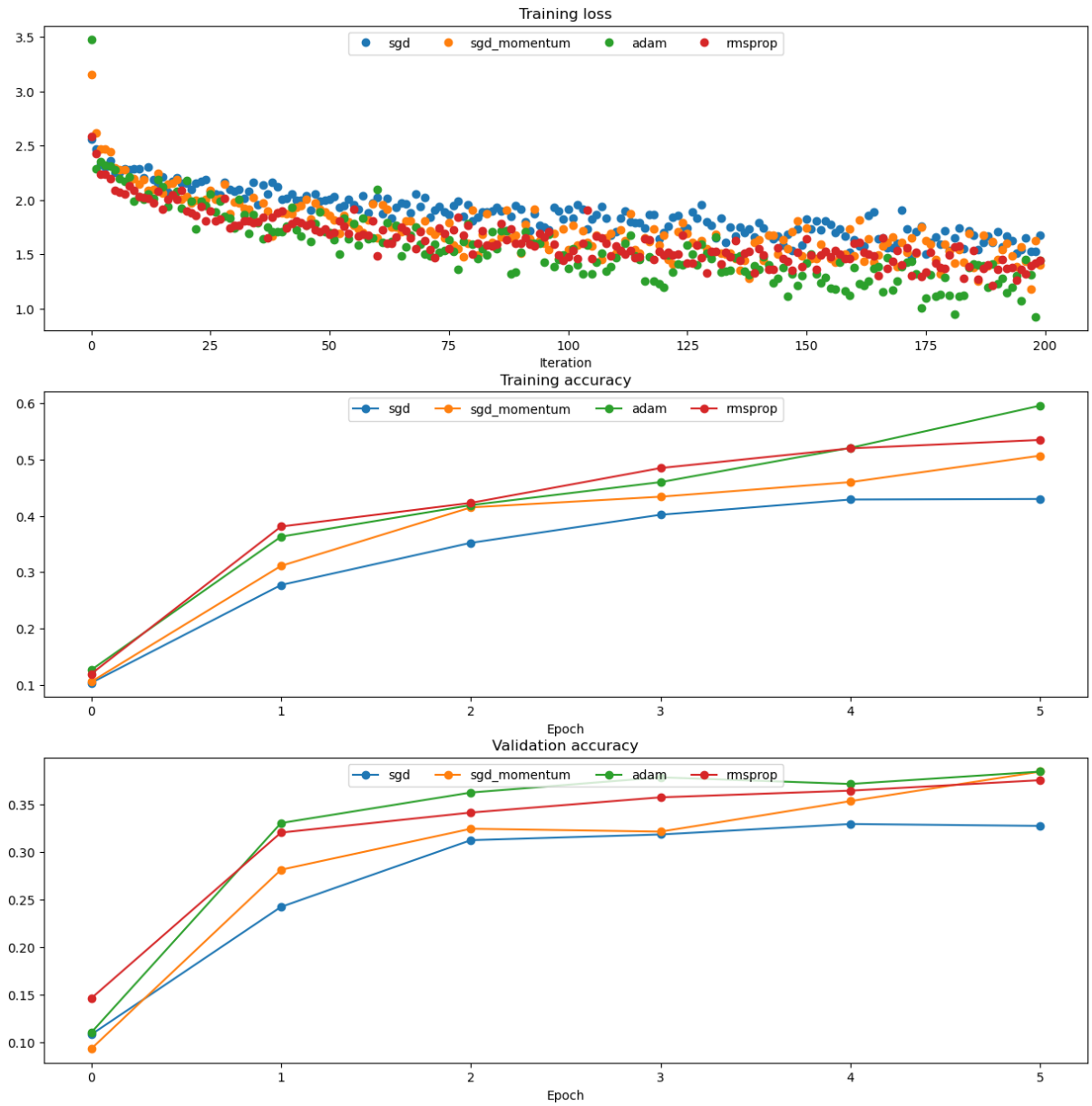
Running with sgd:

(Epoch 5 / 5) train acc: 0.507000; val_acc: 0.384000



Running with adam optimizer:

(Epoch 5 / 5) train acc: 0.535000; val_acc: 0.375000



Train the best FullyConnectedNet:

Best val_acc: 0.514000

Best weight_scale: 0.020000

Best lr: 0.000500

Validation set accuracy: 0.514

Test set accuracy: 0.521

Inline 1-

Sigmoid suffers from the vanishing gradient problem because for very large positive or negative inputs, its output saturates near 1 or 0, making the gradient close to zero. ReLU also has a zero-gradient issue, but only for negative inputs (x ≤ 0), where the gradient becomes

exactly zero and neurons can die. In contrast, Leaky ReLU largely avoids this problem because it allows a small, non-zero gradient for negative inputs, ensuring continued gradient flow during backpropagation.

Inline 2-

Yes, the five-layer network is noticeably harder to train and more sensitive to initialization than the three-layer network. In deeper networks, gradients must pass through more layers during backpropagation, which increases the risk of vanishing or exploding gradients depending on the weight scale. If the initialization is too small, signals shrink as they propagate; if too large, they grow uncontrollably. This effect compounds with depth, making the five-layer network much more sensitive to initialization, while the three-layer network is generally more stable and easier to train.

Inline 3-

In AdaGrad, the cache accumulates the sum of squared gradients over time, so it continuously grows during training. As a result, the denominator (the root of the cache) becomes larger and larger, which shrinks the effective learning rate for each parameter. This causes updates to become very small, slowing learning or even stopping it. Adam does not suffer from this issue in the same way because it uses decaying averages of past gradients exponentially (both first and second moments) instead of a cumulative sum, preventing the denominator from growing indefinitely and allowing the learning rate to remain stable.

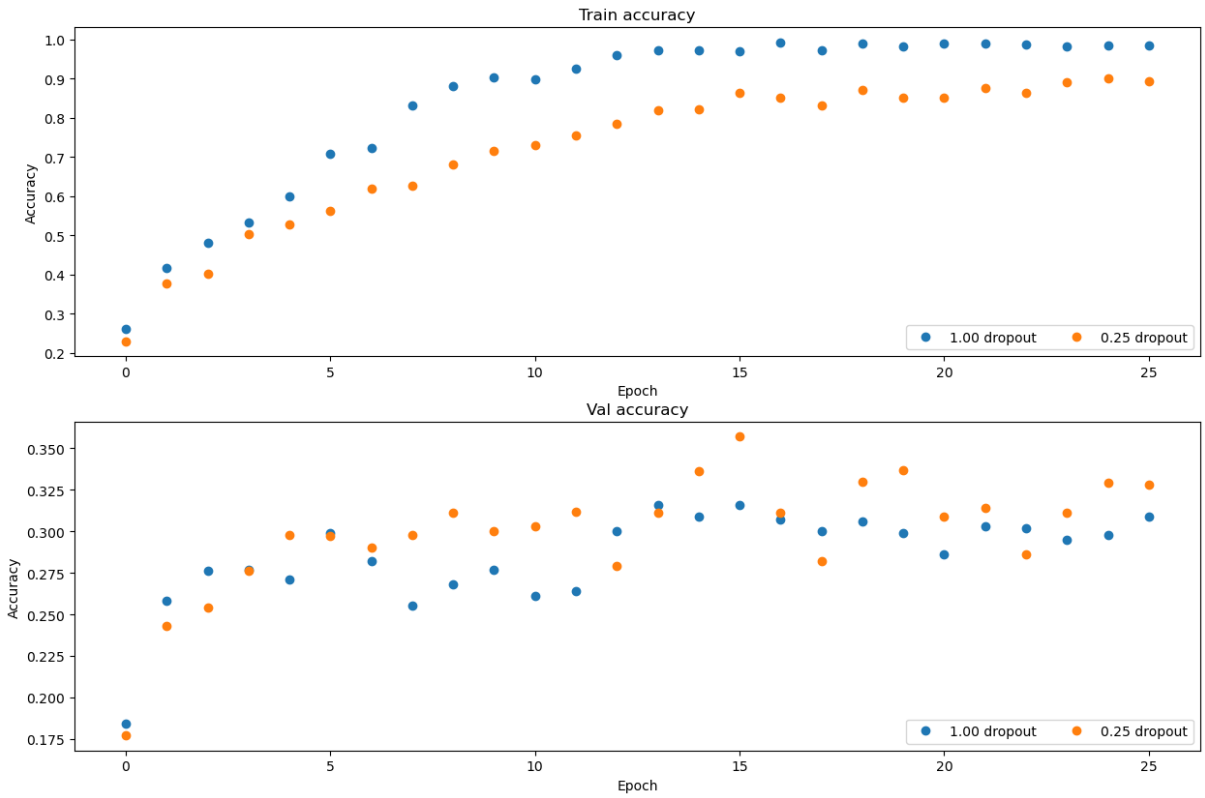
B)

Train two identical nets, one with dropout and one without:

Without dropout:(Epoch 25 / 25) train acc: 0.984000; val_acc: 0.309000

With dropout:(Epoch 25 / 25) train acc: 0.894000; val_acc: 0.328000

Dropout increases the validation accuracy.



Inline 1-

If we do not divide by p in inverted dropout, the activations become smaller during training because only a fraction p of neurons are kept, while the rest are set to zero. This reduces the expected value of the activations by a factor of p, so the network effectively sees weaker signals during training compared to test time (when all neurons are used). As a result, there is a mismatch between training and inference distributions, which can hurt performance and slow learning. Dividing by p compensates for this by keeping the expected activation magnitude consistent between training and testing

Inline 2-

Without dropout, the model typically achieves higher training accuracy but lower validation accuracy, indicating overfitting. With dropout, training accuracy usually decreases slightly due to added noise, while validation accuracy either improves or becomes more stable. This suggests that dropout acts as an effective regularizer by preventing the network from relying too heavily on specific neurons, improving generalization to unseen data.

Inline 3-

If we reduce the size of the hidden layers, we already reduce the model's capacity, lowering the risk of overfitting. Since dropout is also a regularization technique that further reduces effective capacity, keeping the same p (dropout strength) may lead to underfitting. Therefore, we should typically increase p (use less dropout) when the layers are smaller to avoid overly weakening the network and maintaining sufficient learning capacity.

REFERANCES

[1] Google, Gemini (Version 3.1 Pro) [Large Language Model]. Mountain View, CA: Google, 2026. Available: https://gemini.google.com

Description of Use: Assist in writing and debugging Python code. Troubleshoot and debug specific runtime errors, particularly an IndexError caused by array-indexing mismatches in the provided HDF5 dataset. Grammatical reasoning for the network's sequence predictions in Question 2.

APPENDİX

Q1----------

```
“”” 

# -*- coding: utf-8 -*-

"""Q1.ipynb


Automatically generated by Colab.


Original file is located at

    https://colab.research.google.com/drive/1qrKfyzSuOMfLgm8iFZuxc1KlkrF1zmX1

"""


import numpy as np

import h5py

import matplotlib.pyplot as plt


# Load the data

def load_data(filepath):

    with h5py.File(filepath, 'r') as f:

        # Adjust keys if they differ slightly in your actual .h5 file

        train_ims = np.array(f['trainims'])

        train_lbls = np.array(f['trainlbls'])
```

```
        test_ims = np.array(f['testims'])
        test_lbls = np.array(f['testlbls'])

    # Flatten images into 1D vectors for the fully connected network
    num_train = train_ims.shape[0]
    num_test = test_ims.shape[0]

    X_train = train_ims.reshape(num_train, -1)
    X_test = test_ims.reshape(num_test, -1)

    # Normalize pixel values to [0, 1] or [-1, 1] to prevent vanishing gradients with tanh
    X_train = (X_train / 255.0) * 2 - 1
    X_test = (X_test / 255.0) * 2 - 1

    # Ensure labels are shaped correctly as column vectors
    y_train = train_lbls.reshape(-1, 1)
    y_test = test_lbls.reshape(-1, 1)

    # Since we are using tanh for the output neuron, labels should ideally be -1 and 1
    # Assuming original labels are 0 and 1:
    y_train = np.where(y_train == 0, -1, 1)
    y_test = np.where(y_test == 0, -1, 1)

    return X_train, y_train, X_test, y_test

# X_train, y_train, X_test, y_test = load_data('assign2_data1.h5')

class SingleHiddenLayerNN:
```

```
def __init__(self, input_size, hidden_size, output_size=1):
    # Initializing weights with small random values and biases with zeros
    self.W1 = np.random.randn(input_size, hidden_size) * 0.01
    self.b1 = np.zeros((1, hidden_size))
    self.W2 = np.random.randn(hidden_size, output_size) * 0.01
    self.b2 = np.zeros((1, output_size))

def tanh(self, z):
    return np.tanh(z)

def tanh_derivative(self, a):
    return 1.0 - np.power(a, 2)

def forward(self, X):
    self.Z1 = np.dot(X, self.W1) + self.b1
    self.A1 = self.tanh(self.Z1)
    self.Z2 = np.dot(self.A1, self.W2) + self.b2
    self.A2 = self.tanh(self.Z2)
    return self.A2

def compute_loss(self, A2, Y):
    # Mean Squared Error
    m = Y.shape[0]
    return np.mean(np.square(A2 - Y))

def compute_accuracy(self, A2, Y):
    # Classification error metric (percentage of correctly classified images)
    predictions = np.where(A2 >= 0, 1, -1)
```

```
    return np.mean(predictions == Y)

def backward(self, X, Y, learning_rate):
    m = X.shape[0]

    # Gradients for output layer
    dZ2 = (self.A2 - Y) * self.tanh_derivative(self.A2)
    dW2 = (1/m) * np.dot(self.A1.T, dZ2)
    db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)

    # Gradients for hidden layer
    dZ1 = np.dot(dZ2, self.W2.T) * self.tanh_derivative(self.A1)
    dW1 = (1/m) * np.dot(X.T, dZ1)
    db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)

    # Update weights
    self.W1 -= learning_rate * dW1
    self.b1 -= learning_rate * db1
    self.W2 -= learning_rate * dW2
    self.b2 -= learning_rate * db2

def train(self, X_train, Y_train, X_test, Y_test, epochs, batch_size, learning_rate):
    history = {
        'train_mse': [], 'test_mse': [],
        'train_acc': [], 'test_acc': []
    }

    num_samples = X_train.shape[0]
```

```
for epoch in range(epochs):
    # Shuffle data for stochastic mini-batch gradient descent
    indices = np.random.permutation(num_samples)
    X_train_shuffled = X_train[indices]
    Y_train_shuffled = Y_train[indices]

    for i in range(0, num_samples, batch_size):
        X_batch = X_train_shuffled[i:i+batch_size]
        Y_batch = Y_train_shuffled[i:i+batch_size]

        # Forward and backward pass
        self.forward(X_batch)
        self.backward(X_batch, Y_batch, learning_rate)

    # Record error metrics for each epoch separately [cite: 22]
    train_preds = self.forward(X_train)
    test_preds = self.forward(X_test)

    history['train_mse'].append(self.compute_loss(train_preds, Y_train))
    history['test_mse'].append(self.compute_loss(test_preds, Y_test))

    # Storing classification error (1 - accuracy)
    history['train_acc'].append(1 - self.compute_accuracy(train_preds, Y_train))
    history['test_acc'].append(1 - self.compute_accuracy(test_preds, Y_test))

    if (epoch + 1) % 2 == 0:
```

```
            print(f"Epoch {epoch+1}/{epochs} - Train MSE: {history['train_mse'][-1]:.4f} - Test MSE: {history['test_mse'][-1]:.4f}")

        return history

# Assuming X_train, y_train, X_test, y_test are loaded
# input_size = X_train.shape[1]

# Example Hyperparameters to experiment with
# N_hidden = 64
# batch_size = 32
# learning_rate = 0.1
# epochs = 100

# nn = SingleHiddenLayerNN(input_size, N_hidden)
# history = nn.train(X_train, y_train, X_test, y_test, epochs, batch_size, learning_rate)

def plot_learning_curves(history):
    epochs_range = range(1, len(history['train_mse']) + 1)

    plt.figure(figsize=(12, 5))

    # Plot Squared Error [cite: 26]
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, history['train_mse'], label='Train MSE')
    plt.plot(epochs_range, history['test_mse'], label='Test MSE')
    plt.title('Mean Squared Error')
    plt.xlabel('Epoch')
```

```
    plt.ylabel('MSE')
    plt.legend()

    # Plot Classification Error [cite: 26]
    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, history['train_acc'], label='Train Class. Error')
    plt.plot(epochs_range, history['test_acc'], label='Test Class. Error')
    plt.title('Classification Error')
    plt.xlabel('Epoch')
    plt.ylabel('Error Rate')
    plt.legend()

    plt.tight_layout()
    plt.show()

# plot_learning_curves(history)

X_train, y_train, X_test, y_test = load_data('assign2_data1.h5')
# Assuming X_train, y_train, X_test, y_test are loaded
input_size = X_train.shape[1]

# Example Hyperparameters to experiment with
N_hidden = 16
batch_size = 64
learning_rate = 0.1
epochs = 8

nn = SingleHiddenLayerNN(input_size, N_hidden)
```

```
history = nn.train(X_train, y_train, X_test, y_test, epochs, batch_size, learning_rate)
plot_learning_curves(history)

#PART C

# Define our three network capacities
N_low = 4
N_star = 16
N_high = 128

# Keep the optimal parameters from Part A constant
batch_size = 64
learning_rate = 0.1
epochs = 8
input_size = X_train.shape[1]

# 1. Train N_low
print(f"Training N_low = {N_low}...")
nn_low = SingleHiddenLayerNN(input_size, N_low)
history_low = nn_low.train(X_train, y_train, X_test, y_test, epochs, batch_size, learning_rate)

# 2. Train N_star (re-running for a clean comparison)
print(f"\nTraining N_star = {N_star}...")
nn_star = SingleHiddenLayerNN(input_size, N_star)
history_star = nn_star.train(X_train, y_train, X_test, y_test, epochs, batch_size, learning_rate)

# 3. Train N_high
print(f"\nTraining N_high = {N_high}...")
```

```
nn_high = SingleHiddenLayerNN(input_size, N_high)

history_high = nn_high.train(X_train, y_train, X_test, y_test, epochs, batch_size,
learning_rate)


def plot_overlaid_curves(hist_low, hist_star, hist_high, epochs):

    epochs_range = range(1, epochs + 1)


    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    fig.suptitle('Learning Curves: Comparing Network Capacities', fontsize=16)


    # Top-Left: Train MSE

    axs[0, 0].plot(epochs_range, hist_low['train_mse'], label=f'N_low ({N_low})', linestyle='--')

    axs[0, 0].plot(epochs_range, hist_star['train_mse'], label=f'N* ({N_star})', linewidth=2)

    axs[0, 0].plot(epochs_range, hist_high['train_mse'], label=f'N_high ({N_high})', linestyle=':')

    axs[0, 0].set_title('Training Mean Squared Error')

    axs[0, 0].set_xlabel('Epoch')

    axs[0, 0].set_ylabel('MSE')

    axs[0, 0].legend()


    # Top-Right: Test MSE

    axs[0, 1].plot(epochs_range, hist_low['test_mse'], label=f'N_low ({N_low})', linestyle='--')

    axs[0, 1].plot(epochs_range, hist_star['test_mse'], label=f'N* ({N_star})', linewidth=2)

    axs[0, 1].plot(epochs_range, hist_high['test_mse'], label=f'N_high ({N_high})', linestyle=':')

    axs[0, 1].set_title('Testing Mean Squared Error')

    axs[0, 1].set_xlabel('Epoch')

    axs[0, 1].set_ylabel('MSE')

    axs[0, 1].legend()
```

```
    # Bottom-Left: Train Classification Error
    axs[1, 0].plot(epochs_range, hist_low['train_acc'], label=f'N_low ({N_low})', linestyle='--')
    axs[1, 0].plot(epochs_range, hist_star['train_acc'], label=f'N* ({N_star})', linewidth=2)
    axs[1, 0].plot(epochs_range, hist_high['train_acc'], label=f'N_high ({N_high})', linestyle=':')
    axs[1, 0].set_title('Training Classification Error')
    axs[1, 0].set_xlabel('Epoch')
    axs[1, 0].set_ylabel('Error Rate')
    axs[1, 0].legend()

    # Bottom-Right: Test Classification Error
    axs[1, 1].plot(epochs_range, hist_low['test_acc'], label=f'N_low ({N_low})', linestyle='--')
    axs[1, 1].plot(epochs_range, hist_star['test_acc'], label=f'N* ({N_star})', linewidth=2)
    axs[1, 1].plot(epochs_range, hist_high['test_acc'], label=f'N_high ({N_high})', linestyle=':')
    axs[1, 1].set_title('Testing Classification Error')
    axs[1, 1].set_xlabel('Epoch')
    axs[1, 1].set_ylabel('Error Rate')
    axs[1, 1].legend()

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

# Run the plotting function
plot_overlaid_curves(history_low, history_star, history_high, epochs)

#PART D

class TwoHiddenLayerNN:
    def __init__(self, input_size, hidden_size_1, hidden_size_2, output_size=1):
```

```
        # Initialize weights for three layers
        self.W1 = np.random.randn(input_size, hidden_size_1) * 0.01
        self.b1 = np.zeros((1, hidden_size_1))

        self.W2 = np.random.randn(hidden_size_1, hidden_size_2) * 0.01
        self.b2 = np.zeros((1, hidden_size_2))

        self.W3 = np.random.randn(hidden_size_2, output_size) * 0.01
        self.b3 = np.zeros((1, output_size))

    def tanh(self, z):
        return np.tanh(z)

    def tanh_derivative(self, a):
        return 1.0 - np.power(a, 2)

    def forward(self, X):
        self.Z1 = np.dot(X, self.W1) + self.b1
        self.A1 = self.tanh(self.Z1)

        self.Z2 = np.dot(self.A1, self.W2) + self.b2
        self.A2 = self.tanh(self.Z2)

        self.Z3 = np.dot(self.A2, self.W3) + self.b3
        self.A3 = self.tanh(self.Z3)
        return self.A3

    def compute_loss(self, A3, Y):
```

```
    return np.mean(np.square(A3 - Y))

def compute_accuracy(self, A3, Y):
    predictions = np.where(A3 >= 0, 1, -1)
    return np.mean(predictions == Y)

def backward(self, X, Y, learning_rate):
    m = X.shape[0]

    # Gradients for output layer (Layer 3)
    dZ3 = (self.A3 - Y) * self.tanh_derivative(self.A3)
    dW3 = (1/m) * np.dot(self.A2.T, dZ3)
    db3 = (1/m) * np.sum(dZ3, axis=0, keepdims=True)

    # Gradients for second hidden layer (Layer 2)
    dZ2 = np.dot(dZ3, self.W3.T) * self.tanh_derivative(self.A2)
    dW2 = (1/m) * np.dot(self.A1.T, dZ2)
    db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)

    # Gradients for first hidden layer (Layer 1)
    dZ1 = np.dot(dZ2, self.W2.T) * self.tanh_derivative(self.A1)
    dW1 = (1/m) * np.dot(X.T, dZ1)
    db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)

    # Update weights
    self.W1 -= learning_rate * dW1
    self.b1 -= learning_rate * db1
    self.W2 -= learning_rate * dW2
```

```
    self.b2 -= learning_rate * db2
    self.W3 -= learning_rate * dW3
    self.b3 -= learning_rate * db3

def train(self, X_train, Y_train, X_test, Y_test, epochs, batch_size, learning_rate):
    history = {
        'train_mse': [], 'test_mse': [],
        'train_acc': [], 'test_acc': []
    }
    num_samples = X_train.shape[0]

    for epoch in range(epochs):
        indices = np.random.permutation(num_samples)
        X_train_shuffled = X_train[indices]
        Y_train_shuffled = Y_train[indices]

        for i in range(0, num_samples, batch_size):
            X_batch = X_train_shuffled[i:i+batch_size]
            Y_batch = Y_train_shuffled[i:i+batch_size]

            self.forward(X_batch)
            self.backward(X_batch, Y_batch, learning_rate)

        train_preds = self.forward(X_train)
        test_preds = self.forward(X_test)

        history['train_mse'].append(self.compute_loss(train_preds, Y_train))
        history['test_mse'].append(self.compute_loss(test_preds, Y_test))
```

```
        history['train_acc'].append(1 - self.compute_accuracy(train_preds, Y_train))
        history['test_acc'].append(1 - self.compute_accuracy(test_preds, Y_test))

        if (epoch + 1) % 2 == 0:
            print(f"Epoch {epoch+1}/{epochs} - Train MSE: {history['train_mse'][-1]:.4f} - Test MSE: {history['test_mse'][-1]:.4f}")

    return history

# Hyperparameters
H1_size = 16
H2_size = 8
batch_size = 64
learning_rate = 0.1
epochs = 16 # slightly higher to allow the deeper network to converge

print("Training Two-Hidden-Layer Network...")
nn_2layer = TwoHiddenLayerNN(input_size, H1_size, H2_size)
history_2layer = nn_2layer.train(X_train, y_train, X_test, y_test, epochs, batch_size, learning_rate)

# We can reuse the plotting function from part a
plot_learning_curves(history_2layer)

class TwoHiddenLayerNNWithMomentum:
    def __init__(self, input_size, hidden_size_1, hidden_size_2, output_size=1):
        # Initialize weights and biases
        self.W1 = np.random.randn(input_size, hidden_size_1) * 0.01
        self.b1 = np.zeros((1, hidden_size_1))
```

```
        self.W2 = np.random.randn(hidden_size_1, hidden_size_2) * 0.01
        self.b2 = np.zeros((1, hidden_size_2))

        self.W3 = np.random.randn(hidden_size_2, output_size) * 0.01
        self.b3 = np.zeros((1, output_size))

        # Initialize velocity (momentum) terms to zero
        self.v_W1 = np.zeros_like(self.W1)
        self.v_b1 = np.zeros_like(self.b1)

        self.v_W2 = np.zeros_like(self.W2)
        self.v_b2 = np.zeros_like(self.b2)

        self.v_W3 = np.zeros_like(self.W3)
        self.v_b3 = np.zeros_like(self.b3)

    def tanh(self, z):
        return np.tanh(z)

    def tanh_derivative(self, a):
        return 1.0 - np.power(a, 2)

    def forward(self, X):
        self.Z1 = np.dot(X, self.W1) + self.b1
        self.A1 = self.tanh(self.Z1)

        self.Z2 = np.dot(self.A1, self.W2) + self.b2
```

```
        self.A2 = self.tanh(self.Z2)

        self.Z3 = np.dot(self.A2, self.W3) + self.b3
        self.A3 = self.tanh(self.Z3)
        return self.A3

    def compute_loss(self, A3, Y):
        return np.mean(np.square(A3 - Y))

    def compute_accuracy(self, A3, Y):
        predictions = np.where(A3 >= 0, 1, -1)
        return np.mean(predictions == Y)

    def backward(self, X, Y, learning_rate, momentum):
        m = X.shape[0]

        # Gradients
        dZ3 = (self.A3 - Y) * self.tanh_derivative(self.A3)
        dW3 = (1/m) * np.dot(self.A2.T, dZ3)
        db3 = (1/m) * np.sum(dZ3, axis=0, keepdims=True)

        dZ2 = np.dot(dZ3, self.W3.T) * self.tanh_derivative(self.A2)
        dW2 = (1/m) * np.dot(self.A1.T, dZ2)
        db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)

        dZ1 = np.dot(dZ2, self.W2.T) * self.tanh_derivative(self.A1)
        dW1 = (1/m) * np.dot(X.T, dZ1)
        db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)
```

```
        # Update velocities using momentum
        self.v_W3 = momentum * self.v_W3 + learning_rate * dW3
        self.v_b3 = momentum * self.v_b3 + learning_rate * db3

        self.v_W2 = momentum * self.v_W2 + learning_rate * dW2
        self.v_b2 = momentum * self.v_b2 + learning_rate * db2

        self.v_W1 = momentum * self.v_W1 + learning_rate * dW1
        self.v_b1 = momentum * self.v_b1 + learning_rate * db1

        # Update weights using the velocities
        self.W3 -= self.v_W3
        self.b3 -= self.v_b3

        self.W2 -= self.v_W2
        self.b2 -= self.v_b2

        self.W1 -= self.v_W1
        self.b1 -= self.v_b1

    def train(self, X_train, Y_train, X_test, Y_test, epochs, batch_size, learning_rate,
momentum):
        history = {
            'train_mse': [], 'test_mse': [],
            'train_acc': [], 'test_acc': []
        }
        num_samples = X_train.shape[0]
```

```
    for epoch in range(epochs):
        indices = np.random.permutation(num_samples)
        X_train_shuffled = X_train[indices]
        Y_train_shuffled = Y_train[indices]

        for i in range(0, num_samples, batch_size):
            X_batch = X_train_shuffled[i:i+batch_size]
            Y_batch = Y_train_shuffled[i:i+batch_size]

            self.forward(X_batch)
            self.backward(X_batch, Y_batch, learning_rate, momentum)

        train_preds = self.forward(X_train)
        test_preds = self.forward(X_test)

        history['train_mse'].append(self.compute_loss(train_preds, Y_train))
        history['test_mse'].append(self.compute_loss(test_preds, Y_test))
        history['train_acc'].append(1 - self.compute_accuracy(train_preds, Y_train))
        history['test_acc'].append(1 - self.compute_accuracy(test_preds, Y_test))

        if (epoch + 1) % 2 == 0:
            print(f"Epoch {epoch+1}/{epochs} - Train MSE: {history['train_mse'][-1]:.4f} - Test
MSE: {history['test_mse'][-1]:.4f}")

    return history

# Hyperparameters
```

```
H1_size = 16

H2_size = 8

batch_size = 64

learning_rate = 0.1

momentum_alpha = 0.5  # New momentum term

epochs = 15


print(f"Training Two-Hidden-Layer Network with Momentum
(alpha={momentum_alpha})...")

nn_momentum = TwoHiddenLayerNNWithMomentum(input_size, H1_size, H2_size)

history_momentum = nn_momentum.train(X_train, y_train, X_test, y_test, epochs,
batch_size, learning_rate, momentum_alpha)


# Plot the results

plot_learning_curves(history_momentum)


“”“
```

Q2----------

```
“”“

# -*- coding: utf-8 -*-

"""Q2.ipynb


Automatically generated by Colab.


Original file is located at

    https://colab.research.google.com/drive/1TXAPknSnwgWDdifv_pxR-h-3vn0Az-Vr

"""


import numpy as np
```

```
import h5py
import matplotlib.pyplot as plt

def load_text_data(filepath):
    with h5py.File(filepath, 'r') as f:
        # Load inputs and targets (labels) and subtract 1 for 0-based indexing
        trainx = np.array(f['trainx'], dtype=np.int32) - 1
        traind = np.array(f['traind'], dtype=np.int32) - 1
        valx = np.array(f['valx'], dtype=np.int32) - 1
        vald = np.array(f['vald'], dtype=np.int32) - 1
        testx = np.array(f['testx'], dtype=np.int32) - 1
        testd = np.array(f['testd'], dtype=np.int32) - 1

    # Squeeze targets to ensure they are 1D arrays for indexing
    return trainx, traind.squeeze(), valx, vald.squeeze(), testx, testd.squeeze()

class TrigramPredictorNN:
    def __init__(self, vocab_size, D, P):
        self.vocab_size = vocab_size
        self.D = D
        self.P = P

        # Initialize weights and biases as random Gaussian variables of std 0.01
        self.R = np.random.randn(vocab_size, D) * 0.01
        self.W1 = np.random.randn(3 * D, P) * 0.01
        self.b1 = np.random.randn(1, P) * 0.01
        self.W2 = np.random.randn(P, vocab_size) * 0.01
        self.b2 = np.random.randn(1, vocab_size) * 0.01
```

```
    # Initialize velocities for momentum
    self.v_R = np.zeros_like(self.R)
    self.v_W1 = np.zeros_like(self.W1)
    self.v_b1 = np.zeros_like(self.b1)
    self.v_W2 = np.zeros_like(self.W2)
    self.v_b2 = np.zeros_like(self.b2)

def sigmoid(self, z):
    # Clip z to prevent overflow
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative(self, a):
    return a * (1.0 - a)

def softmax(self, z):
    # Subtract max for numerical stability
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

def forward(self, X):
    # X is shape (batch_size, 3) containing word indices
    # Embedding lookup: output shape is (batch_size, 3, D)
    self.embedded = self.R[X]

    # Flatten embeddings for the hidden layer: (batch_size, 3 * D)
    self.flattened_emb = self.embedded.reshape(X.shape[0], -1)
```

```
    # Hidden layer with Sigmoid
    self.Z1 = np.dot(self.flattened_emb, self.W1) + self.b1
    self.A1 = self.sigmoid(self.Z1)

    # Output layer with Softmax
    self.Z2 = np.dot(self.A1, self.W2) + self.b2
    self.A2 = self.softmax(self.Z2)

    return self.A2

def compute_cross_entropy(self, A2, Y):
    m = Y.shape[0]
    # Clip predictions to prevent log(0)
    A2_clipped = np.clip(A2, 1e-9, 1 - 1e-9)
    # Select the probability assigned to the true class
    correct_class_probs = A2_clipped[np.arange(m), Y]
    return -np.mean(np.log(correct_class_probs))

def backward(self, X, Y, learning_rate, momentum):
    m = X.shape[0]

    # Gradient of Softmax + Cross Entropy
    dZ2 = self.A2.copy()
    dZ2[np.arange(m), Y] -= 1
    dZ2 /= m

    # Hidden to Output Gradients
```

```
dW2 = np.dot(self.A1.T, dZ2)
db2 = np.sum(dZ2, axis=0, keepdims=True)

# Hidden Layer Gradients
dA1 = np.dot(dZ2, self.W2.T)
dZ1 = dA1 * self.sigmoid_derivative(self.A1)

dW1 = np.dot(self.flattened_emb.T, dZ1)
db1 = np.sum(dZ1, axis=0, keepdims=True)

# Embedding Layer Gradients
d_flattened_emb = np.dot(dZ1, self.W1.T)
d_embedded = d_flattened_emb.reshape(m, 3, self.D)

# We need to accumulate the gradients for the specific word embeddings used
dR = np.zeros_like(self.R)
np.add.at(dR, X, d_embedded)

# Apply Momentum updates
self.v_W2 = momentum * self.v_W2 + learning_rate * dW2
self.v_b2 = momentum * self.v_b2 + learning_rate * db2
self.v_W1 = momentum * self.v_W1 + learning_rate * dW1
self.v_b1 = momentum * self.v_b1 + learning_rate * db1
self.v_R = momentum * self.v_R + learning_rate * dR

# Update weights
self.W2 -= self.v_W2
self.b2 -= self.v_b2
```

```
        self.W1 -= self.v_W1
        self.b1 -= self.v_b1
        self.R -= self.v_R

    def train(self, X_train, Y_train, X_val, Y_val, epochs, batch_size, learning_rate,
momentum):
        history = {'train_loss': [], 'val_loss': []}
        num_samples = X_train.shape[0]
        best_val_loss = float('inf')
        patience_counter = 0

        for epoch in range(epochs):
            indices = np.random.permutation(num_samples)
            X_train_shuffled = X_train[indices]
            Y_train_shuffled = Y_train[indices]

            for i in range(0, num_samples, batch_size):
                X_batch = X_train_shuffled[i:i+batch_size]
                Y_batch = Y_train_shuffled[i:i+batch_size]

                self.forward(X_batch)
                self.backward(X_batch, Y_batch, learning_rate, momentum)

            # Evaluate after epoch
            train_preds = self.forward(X_train)
            val_preds = self.forward(X_val)

            train_loss = self.compute_cross_entropy(train_preds, Y_train)
```

```
        val_loss = self.compute_cross_entropy(val_preds, Y_val)

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)

        print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_loss:.4f} - Val Loss: {val_loss:.4f}")

        # Early stopping based on validation data
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= 3: # Stop if validation loss doesn't improve for 3 epochs
                print(f"Early stopping triggered at epoch {epoch+1}.")
                break

    return history

# Assuming data is loaded
trainx, traind, valx, vald, testx, testd = load_text_data('assign2_data2.h5')

vocab_size = 250
batch_size = 200
learning_rate = 0.15
momentum = 0.85
max_epochs = 50
```

```
configs = [(32, 256), (16, 128), (8, 64)] # (D, P) pairs
trained_models = {}

plt.figure(figsize=(10, 6))

for D, P in configs:
    print(f"\n--- Training with D={D}, P={P} ---")
    model = TrigramPredictorNN(vocab_size, D, P)
    hist = model.train(trainx, traind, valx, vald, max_epochs, batch_size, learning_rate,
momentum)

    trained_models[(D, P)] = model

    plt.plot(hist['val_loss'], label=f'Val Loss (D={D}, P={P})')

plt.title('Validation Cross-Entropy Loss for Different Architectures')
plt.xlabel('Epoch')
plt.ylabel('Cross-Entropy Loss')
plt.legend()
plt.show()

import h5py

# Load the vocabulary words from the h5 file and decode them
with h5py.File('assign2_data2.h5', 'r') as f:
    # Decode byte strings to regular python strings
    vocab_words = [word.decode('utf-8') for word in f['words'][:]]
```

```
# Assuming 'trained_models' from Part (a) is still in memory,
# let's select the model with the most capacity (D=32, P=256)
best_model = trained_models[(32, 256)]

# 1. Pick 5 random sample indices from the test data
np.random.seed(42) # For reproducibility
num_test_samples = testx.shape[0]
sample_indices = np.random.choice(num_test_samples, 5, replace=False)

sample_trigrams = testx[sample_indices]
actual_fourth_words = testd[sample_indices]

# 2. Generate predictions for the fourth word using the trained model
predictions = best_model.forward(sample_trigrams)

# 3. List the top 10 candidates for each of the 5 samples with ACTUAL WORDS
print("Top 10 Predictions for 5 Sample Trigrams:\n" + "="*55)

for i in range(5):
    trigram_ids = sample_trigrams[i]
    actual_word_id = actual_fourth_words[i]
    probs = predictions[i]

    # Translate the trigram IDs to words
    trigram_words = [vocab_words[idx] for idx in trigram_ids]
    actual_word_str = vocab_words[actual_word_id]

    # Get indices of the top 10 probabilities in descending order
```

```
    top_10_indices = np.argsort(probs)[::-1][:10]
    top_10_probs = probs[top_10_indices]

    print(f"Sample {i+1}:")
    print(f"  Input Trigram: '{trigram_words[0]} {trigram_words[1]} {trigram_words[2]}'")
    print(f"  Actual 4th Word: '{actual_word_str}'")
    print(f"  Top 10 Predicted Candidates:")

    for rank, (idx, prob) in enumerate(zip(top_10_indices, top_10_probs)):
        predicted_word = vocab_words[idx]
        # Mark the correct prediction with an asterisk if it's in the top 10
        marker = " <--- ACTUAL" if idx == actual_word_id else ""
        print(f"    {rank+1:2d}. {predicted_word:15s} | Prob: {prob:.4f}{marker}")
    print("-" * 55)

"""
```